Keyboard to Morse Code Translator

Final Project Report ENGS 31 (20X)

Authors: Sarah Korb, Ellie Baker

Abstract

For our final project we created a keyboard to Morse code converter. Our device can receive up to 257 characters of input from the computer keyboard. The message is held until a total of 257 characters are received or the enter key is pressed. Then the message is displayed as Morse code through an LED light, which flashes to indicate dashes and dots. Dots are approximately 0.5 seconds long, dashes are 1.5 seconds long, and sequential dashes and dots are separated by 0.5 seconds (with no LED). The hexadecimal values of the typed characters are also displayed on the 4-digit display as they are typed and received.

Table of Contents

1. Introduction…….3
2. Design Solution……..3
   - 2.1 Specifications……...3
   - 2.2 Operation Instructions……4
   - 2.3 Theory of Operation……5
     - 2.31 UART/Serial Receiver…….6
     - 2.32 Queue……...7
     - 2.33 7-Segment Display……...10
     - 2.34 ASCII to Morse Converter……...10
     - 2.35 Top level shell…….12
   - 2.4 Construction and Debugging……13
3. Evaluation ……15
4. Conclusion……16
5. Acknowledgements……16
6. References…….17

1. Introduction

Our final project is a keyboard to Morse code converter, which takes keyboard input from a command line and then visibly displays the message in Morse code using a flashing LED light on the FPGA.

2. Design Solution

2.1 Specifications

Our circuit receives serial ASCII input from the keyboard. The ASCII characters are received at the serial port on the FPGA, which is connected to the computer with a USB cord; this is the same cord/port that is used to power the FPGA. The ASCII characters are then translated into their respective Morse code after the 'enter' key is pressed or after 257 characters are received. The Morse code is visually outputted with an LED light on the FPGA. A keyboard and USB cable connection to the serial port are necessary for the circuit's inputs, and the LED light is necessary for the Morse output. The Master clock into the circuit is 100MHZ, but this is divided down to 10 MHz using a clock divider in the project's top level shell file.

We decided on a dot time of 0.5 seconds so that the Morse code could be easily observed and read for beginners. The Morse code is output using standard Morse code timing ratios:

Dot = 0.5 seconds

Dash = 3xDot time

Space between Dashes and Dots = Dot time

Space between characters = 3xDot time

Space between words = 7xDot time

With a Dot speed of 2 dots per second, this Morse code translator outputs approximately 5 words per minute. A 'blackbox' depiction of our design can be seen in Figure 2.1. of the Appendix, which shows its overall inputs and outputs. A more detailed top-level block diagram can be seen in Figure 2.2 of the Appendix.

2.2 Operation Instructions

To use the Morse code translator, PuTTy needs to be installed on the computer. First, connect the FPGA's serial input port to the computer using a USB cable. Next, open a PuTTy terminal and input the following settings:

1. Under the Session tab:

    Connection Type: Serial (*The serial line must be the address to the usb serial port.)*
    Speed: 115200

2. Under the Terminal tab:

    Local echo: Force on (*This will allow you to see what you type in the PuTTy terminal)*

3. Under the Serial tab:

    Speed: 115200
    Data bits: 8
    Stop bits: 1
    Parity: None
    Flow Control: None

Once these settings are completed and a new PuTTy terminal window is open, type in a message up to 257 characters. If the message is less than 257 characters, hit the enter key to begin the translation. If the message is equal to 257 characters, the translation will begin automatically. The Morse code output should be visible as a flashing, green LED light on the FPGA. During this time, no new keyboard input will be stored for output; the translator must finish outputting its current message first.

2.3 Theory of Operation

Our design is comprised of many components, but can be generalized into the following overview:

An ASCII encoded character is received in a serial stream from the keyboard. The bits are shifted and stored using an 8-bit serial-to-parallel register, and then stored in a FIFO queue. The circuit continues receiving serial input and repeating this process until either 257 characters have been stored in the queue, or an 'ENTER' character is received.

The queue then reads out the 8-bit ASCII input in the same order that it entered the queue; each 8-bit ASCII code enters a lookup table, where its corresponding 7-bit Morse code representation is then output.

The Morse code bit representations from the lookup table are output to a parallel to serial converter. As the bits are shifted, the shift register pauses on the dash and dot bits for specified

amounts of time so the LED light is turned on for the corresponding dash or dot time. More detail about how the Morse code bit representation works can be found in our explanation of our serial to parallel shift register below (*see below 2.34 ASCII-to-Morse converter, Morse to LED controller*)**.**

Using abstraction, we can separate our design into 4 main sections: UART/serial receiver, Queue, ASCII-to-Morse converter, and the 7-segment display. These are all joined in a top level shell file [redacted]. Each of these sections has its own datapath and controller. The following explains these sections and their component blocks in greater detail.

2.31 UART/Serial Receiver: *See Figure 2.3 of the Appendix*

This is the first section of our design, and is where the serial keyboard input is first received

SerialRX Block:

The SerialRX block receives a serial stream of bits through a USB connection to a computer using puTTY. The serial stream is synchronized to a 10MHz clock (used throughout the design project) by a double-flop synchronizer process. Triggered by a low-true start bit, an edge counter in the SerialRX controller shifts the synchronized serial input into a 10-bit shift register that converts from serial to parallel. The edge counter is designed based on a given baud rate so that each serial input shifted in is a new bit of a 10 bit ASCII value. After 10 bits are shifted into the shift register, load_en is asserted and bits 9 down to 2 of those stored in the shift register are loaded into a load register (the 10th bit is the stop bit and the 1st bit is the start bit). The load

register outputs the 8 bit data as rx_data. Rx_data represents the ASCII code corresponding to one letter.

2.32 Queue: *See Figure 2.2 of the Appendix (Top Level)*

Eight-bit ASCII character code is written into a queue that holds the information, forming a buffer. Creating this buffer ensures timing control for ASCII character processing, which is important to the third part of the design (ASCII to Morse conversion and LED output). The queue can hold 257 8-bit ASCII characters at one time. To process ASCII code, characters are read from the queue. When all characters have been read the queue clears then moves to an idle state.

257x8 bit buffer (Queue datapath):

The queue datapath receives an 8-bit ASCII character and assigns it to a location in the queue register file based on a write address input that is generated in the controller. This process is synchronous, and only occurs when the signal write_en is asserted. Write_en is an output of the controller that is only asserted in the write state. The datapath output is an 8-bit ASCII character read from the queue based on a read address input that is generated in the controller. This process only occurs when the signal read_en is asserted. Read_en is an output of the controller that is only asserted in the read state.

Queue Controller:

Write Address Generator: Write_addr is an output to the Queue datapath. It is generated in a synchronous process that increments the write address based on the signal inc_w, which is

asserted in the write state. The write address is cleared whenever clr_w is asserted, which happens in the clear state.

Read Address Generator: Read_addr is an output to the Queue datapath. It is generated in a synchronous process that increments the read address based on the signal inc_r, which is asserted in the read state. The read address is cleared whenever clr_r is asserted, which happens in the clear state.

Full and Empty: The read and write addresses initially generated in the controller are intermediate signals write_addrint and read_addrint respectively. Each signal is one bit larger than the outputs passed to the datapath from the controller as write_addr and read_addr. This design allows for the following logic, which determines whether the queue is full or empty. The queue is full when the most significant bit of write_addrint is 1 and the most significant bit of read_addrint is 0 while the remaining bits of each signal are equal to one another. When this occurs, an intermediate signal full is set to 1, which causes a transition from write state to read state in the FSM. Since write_addrint is not reset in read, read_addrint and write_addrint will only be equal to one when the queue is empty, or after all characters have been read from the queue and the values held in the queue can be written over.

FSM:

The FSM generates signals sent to the controller read and write address generators that increment and clear the read and write addresses. The FSM also generates output signals which are passed to the Queue datapath and enable the register, which holds up to 257 ASCII characters. A detailed diagram of the Queue FSM can be found in Appendix Figure 3.2. The signals full, readdone, and move derived from logic described in the Full and Empty section above move the

controller between states, as do input signals rx_done_tick and done, which come from the SerialRX and Morse2LED components respectively. The FSM starts in the idle state. From the idle state it is possible to move into either a read state or a write state. If done is one and all characters have not already been read from the queue, the controller moves into read state from idle. Otherwise, if rx_done_tick is one and the queue is not full, the controller moves into the write state. In the write state, a write address increment signal inc_w is asserted and write_en is asserted. The controller only stays in the write state for one clock cycle, after which it moves back to idle or to read if the queue is full (full= 1) or the message is over (finall=1). In this manner, the ASCII characters of an entire message are written into the queue, according to timing (rx_done_tick) determined by the SerialRX block. In the read state, the read address increment signal inc_r is asserted and read_en is asserted. The controller stays in the read state for only one clock cycle, after which it moves to the idle state, or if all of the characters in the queue have been read out (move = 1) the controller moves to the clear state. In this manner the ASCII characters of an entire message are read out of the queue, according to timing (done signal) determined by the Morse2LED component. In the clear state the clear read and write address signals clr_r and clr_w are asserted. The controller stays in the clear state for one clock cycle then moves to idle. The controller will remain in the idle state until rx_done_tick is asserted indicating a new message can be read into the queue.

2.33 7-Segment Display(Mux7seg): *See Figure 2.5 of the Appendix*

This is where the hexadecimal representation of the incoming ASCII characters from the keyboard are displayed. 8-bit ASCII character information is split into bits 4 to 1 and 8 to 4 and assigned to inputs y1 and y2 respectively. Output and control anodes that rapidly "enable" digits 1, 2, 3, and 4 of the display in succession so to the human eye all 4 digits appear to be displayed at once. Seg output is the decoded binary which uses cathodes to represent a hexadecimal character to be displayed, and dp output determines whether decimal points are displayed. The component consists of a counter to drive the rapidly changing an output, a clock divider that slows the 10MHz clock to a slower CE clock used to time outputs from Mux7seg, a multiplexer that assigns inputs y1, y2, y3 or y4 to an intermediary signal muxy depending on the current an value, and a seven segment decoder that takes in the 4-bit intermediary signal muxy and outputs 7-bit binary as seg.

2.34 ASCII to Morse Converter : *See Figure 2.4 of the Appendix*

This is where the ASCII character codes from the queue are translated into Morse code, which is then used to light the LED output.

ASCII to Morse Lookup table:

The lookup table takes an 8-bit ASCII input, and outputs a 7-bit Morse code output corresponding to the Morse code representation of this letter. Our Morse code bit translation represents dashes as 1's and dots as 0's. To make all Morse code bit representations uniform, each sequence of 1's and 0's is first padded with a start bit '1' and then further padded with 0's until the sequence is 7 bits long. Our design is able to translate and represent A-Z, 0-9 and

spaces; all other characters are automatically translated as a space, and both uppercase and lowercase letters are given the same translation. Once a 7-bit Morse code is output, the internal signal 'Morse_received' is toggled to '1' and sent to the controller.

Morse to LED datapath:

After the LUT translates the ASCII code to a 7-bit Morse code representation, it is first checked to see if it is "0000000", which is representative of a space. If this is the case, the signal 'space_signal' goes high and is sent to the controller. The 7-bits are then loaded into a parallel to serial shift register, which is enabled by signals from the controller. Beginning with the padding of 0's, the serial bits are shifted out; the signal 'dash' is assigned and re-assigned to each bit as they are shifted out, and dash is then sent to the controller. Once dash = 1 for the first time, we have hit the 'start' bit, which tells the controller that the upcoming bits are representative of dashes and dots. The Datapath receives an LED_en signal from the controller, which is then output as LED_out to the LED light on the FPGA.

Morse to LED controller:

The controller is what determines whether the LED light is to be turned on or off, and for how long. A detailed diagram of the controller can be found in Appendix Figure 3.3 It begins in an idle state. Once it receives a high 'Morse_received' signal from the datapath, it either moves to the load state (when space_detected is false) or a space state (when space_detected is true). In the space state, a timer is started and has a tc signal that goes high after the time for a space has passed. The controller then returns to idle.

In the case of a character, the controller enables the parallel load register and then enables the shift register, which begins to shift though the padding of 0's. Once the controller receives a high 'dash' signal from the datapath, it enters the read state, where it will begin reading input as dots and dashes. 0's take the controller to the dot state, where the LED is enabled; a timer with a tc signal that goes high after the specified dot time returns the controller to read. Likewise, 1's take the controller to a dash state, where an identical process occurs but with the timer set for the length of a dash.In between these dots and dashes, the controller waites in a 'waiting' state, which disables the LED and distinguishes dots from dashes in the LED output. The time spent in this waiting state is controlled by a timer with the length of a dot.

Once all 7 bit's have been shifted, which is noted by a shift counter, the controller returns to read and then enters another waiting state, this time to indicate the space between characters. The time spent in this waiting state is controlled by a timer with the length of 3x a dot. It then enters a done state which monopulses a 'done' signal sent to the Queue, indicating it is ready for another character, and then returns to idle.

2.35 Top- level Shell:

This connects all 4 sections of the project as components in a single, top-level shell file. This shell file also contains a clock divider which divides the master 100MHz clock down to 10MHz. This slower clock is passed as the 'clk' input to all the components.

2.4 Construction and Debugging

We dedicated a lot of time to planning our design using block diagrams and FSM diagrams before beginning any code. We split our design into four main sections, as can be seen in Figure 2.2 of the Appendix. We further split each of these sections into a datapath and controller, and began writing the code. Using testbenches, each datapath and controller were tested separately to make sure they had the correct behavior and output when given a predetermined set of inputs. Afterwards, they were connected in a shell file and tested as a whole unit. Finally, each of the four main sections were included as components of a top-level Project shell file, which was tested using a testbench, and then tested in hardware.

We first began with the SerialRX block of our design, which is where the keyboard input is first processed. We relied heavily on a previous year's Lab5 in the construction of this portion; it helped us build the synchroniser, Serial RX datapath and controller. We then worked on the final section of the design, where the ASCII code was to be translated and output as a Morse code LED. This required significantly more time than the prior section; we began with the datapath and controller that would process a Morse code bit representation and output the LED. Once we had assured these portions worked separately using testbenches, we joined them in an upper-level shell file, where we testbenched them together. We then added the ASCII-to-Morse LUT into this shell, and configured the ASCII input of the shell to enter this LUT first. Finally, we wrote and tested the datapath and controller for the Queue, which were then joined in a shell

file and tested as a whole. The 7-segment display was a pre-existing file taken from previous labs, and it was added as the last component in our top level project file.

As we used a testbench for each datapath, controller and component shell, debugging occurred throughout the process of construction. Some of the most common issues were internal signals either being a clock period too early or late, which greatly affected the functionality of our FSM machines. In addition, we encountered issues where the keyboard data was received, but never made it any farther than the initial serial port. This was ultimately due to a mistake in port mapping in the top-level shell file.

The most difficult debugging came when we were testing the design in hardware as we didn't have the testbench simulation displaying any signals. For example, the controller of the ASCII-to-Morse block was dependent on multiple tc signals to exit the dash, dot and various waiting states. If these tc signals toggled at the wrong times, or didn't toggle at all, the FSM would be stuck in a dash or dot state, resulting in the LED light permanently on. To combat this, we tested the ASCII-to-Morse section alone in hardware, giving it a default ASCII value of an 'A' to process and output in Morse. We found we had forgotten to include the tc signals in the sensitivity list of our timer processes, but testing this block alone in hardware was imperative to coming to this conclusion.

3. Evaluation

We believe our design is efficient and easily understood. In the process of constructing our design, one of the first issues we faced was how to represent Morse code using bits. Initially, we wanted to use a sequence of 1's to represent dashes and dots, with dashes being a sequence of three times the length of dots (i.e dots would be '11' and dashes would be '111111'). Following this logic, spaces in between dashes and dots would be represented as a sequence of 0's. However, we quickly realized that the Morse code bit representations for each letter would vary greatly in length, and would also be fairly long in length. We also didn't know how to make the clock iterate through these sequences at a slow enough rate so that the LED output was clearly visible and understandable. Therefore, we altered our Morse code translation to represent dashes as 1s, dots as 0s, and use an FSM to control the length of the LED output. We feel this representation is more instinctive. This design allowed for a uniform length of 7 for all Morse code bit representations, and allowed us to use the same clock for all the components.

There are also many ways our design could have been improved and developed. For example, our Queue stores an initial value of "11111111", which we translate to the equivalent of a space. This causes a slight delay in the output of the very first message, as a 'pseudo-space' is displayed (i.e a period of no LED light) before the first character. Furthermore, we would explore ways to add functionality to our design, such as a way to vary the speed of the Morse code output, an audio Morse output, or a function that echoes the Morse code message back as characters in the computer terminal.

4. Conclusion

Our design successfully translates keyboard input to a visible Morse code output at a readable rate; it includes all of the functionality and specification objectives with which we began the project. Our design and construction process used testbench simulations of every datapath, controller and component to simplify the debugging process. For future groups, we suggest following a similar process of using testbenches at every step possible; it makes the debugging process along the way a lot more efficient. It also familiarized us with the components and subcomponents, and the possible errors that could cause their misbehavior. This was extremely valuable when testing in hardware, where we could not view the internal signals in the same way that the testbenches allowed.

5. Acknowledgement

We would like to thank Professor Hansen for his advice in designing our project and for helping us during the debugging process in both software and hardware. We would also like to thank Ben C. for his dedication, patience, helpful insight, and for always being on call when our code wasn't functioning properly.

The project was divided evenly between the two of us. We talked through and designed each of the block diagrams together, and then split up the coding responsibilities. Ellie was responsible for the Queue, 7-segment display, the SerialRX datapath, the top-level shell and their respective

testbenches. Sarah was responsible for the ASCII-to-Morse block, the SerialRX controller, their respective testbenches and setting up the PuTTy interface. However, we were still familiar enough with eachother's sections that testing and debugging errors was often done together; a fresh set of eyes to look over faulty code proved to be very valuable. We would also consult each other on design advice, and worked together to port map and connect all the components.

6. References

We used previous labs, lecture slides and information in the Canvas modules to help us complete this project. In specific, a previous year's Lab5 (entitled *Serial Communications*) was used as a scaffold for our SerialRX block. We also used a previous year's Final project report (which was uploaded to Canvas as a source) , entitled *Keyboard to Morse Code Converter* to guide us in our design process.